

Автономная некоммерческая организация среднего профессионального
образования «СТАРТ КОЛЛЕДЖ»

Разработка автоматизированной информационной системы для фитнес-
центра

Работу выполнил:

Румянцев Роман Антонович

Научный руководитель:

Сотникова Сабрина Дмитриевна

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. ОБЩАЯ ЧАСТЬ.....	6
1.1 Анализ объектной области.....	6
1.2 Анализ предметной области.....	8
1.3 Описание средств и среды разработки.....	10
1.4 Обзор существующих вариантов программных продуктов.....	12
2. СПЕЦИАЛЬНАЯ ЧАСТЬ.....	16
2.1 Описание требований к информационной системе.....	16
2.2 Функциональный сценарий.....	18
2.3 Проектирование структуры данных.....	20
2.4 Описание ролей и прав доступа.....	22
2.5 Пользовательские сценарии.....	24
2.6 Описание интерфейсов.....	25
2.7 Тестирование и отладка.....	27
ЗАКЛЮЧЕНИЕ.....	30
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	32
ПРИЛОЖЕНИЕ А. ЛИСТИНГИ ИСХОДНОГО КОДА.....	34
ПРИЛОЖЕНИЕ Б. ИНТЕРФЕЙСЫ СИСТЕМЫ.....	43

ВВЕДЕНИЕ

Фитнес-центр каждый день работает с большим потоком информации. Клиенты покупают абонементы, записываются на занятия, уточняют расписание и обращаются к администраторам. Тренеры проводят групповые и индивидуальные тренировки, следят за своим графиком и количеством записанных людей. Администратор оформляет клиентов, контролирует оплаты, добавляет тренировки и должен быстро понимать, что происходит в клубе. Если все эти данные лежат в разных таблицах, бумажных журналах или переписках, работа начинает тормозить. Появляются ошибки, теряются сведения об оплатах, расписание приходится перепроверять вручную.

Актуальность моего проекта связана именно с этой проблемой. Фитнес-центру нужна понятная информационная система, где основные процессы собраны в одном месте. Администратор должен видеть клиентов, тренеров, абонементы, платежи и ближайшие занятия. Тренеру нужен доступ к своему расписанию. Клиенту нужен личный кабинет, где можно посмотреть данные об абонементе и записях на тренировки. Такая система не заменяет работу сотрудников, но убирает лишнюю ручную рутину и помогает быстрее находить нужную информацию.

Проблема исследования состоит в том, что ручной учет плохо подходит для фитнес-центра, где данные постоянно меняются. Абонемент может закончиться, клиент может записаться на новое занятие, тренер может вести несколько направлений, зал может быть занят в нужное время. При ручной работе сотруднику приходится держать в голове слишком много деталей или каждый раз сверяться с разными источниками. Из-за этого растет риск ошибки: клиента могут записать не на то занятие, оплату могут забыть отметить, а руководитель получит устаревшую картину по работе клуба.

Объектом дипломной работы является деятельность фитнес-центра, связанная с обслуживанием клиентов, работой тренеров, расписанием занятий и учетом абонементов. Предметом дипломной работы является автоматизация этих процессов с помощью веб-приложения, где есть регистрация пользователей, ролевой доступ, учет клиентов и тренеров, планы абонементов, платежи, тренировочные залы, занятия и записи клиентов.

Цель дипломной работы — разработать автоматизированную информационную систему для фитнес-центра. Система должна хранить данные в единой базе, разделять пользователей по ролям и давать каждому участнику свой набор функций через веб-интерфейс.

Для достижения цели нужно разобрать работу фитнес-центра, определить основные роли пользователей, изучить похожие программные продукты, сформулировать требования к системе, продумать структуру данных, описать интерфейсы и пользовательские сценарии, настроить разграничение доступа, а также проверить работу основных функций приложения.

В работе применяются анализ предметной области, сравнение программных продуктов, проектирование структуры данных, объектно-ориентированный подход, разработка веб-интерфейса и функциональное тестирование. Практическая часть выполнена на языке Python с использованием фреймворка Django. Для хранения данных используется SQLite, для оформления страниц — Bootstrap, а для административной части — стандартная панель Django.

Практическая значимость проекта заключается в том, что полученное приложение можно использовать как основу для автоматизации небольшого или среднего фитнес-центра. Оно помогает хранить данные в одном месте, быстрее получать сводную информацию, разделять права доступа и уменьшать количество ошибок при работе с расписанием, абонементом и платежами.

Дипломная работа состоит из введения, общей части, специальной части, заключения, списка используемых источников и приложений. В общей части

рассматривается деятельность фитнес-центра, предметная область, средства разработки и существующие аналоги. В специальной части описаны требования, сценарии работы, база данных, роли пользователей, интерфейсы и тестирование.

Источниками информации для дипломной работы стали официальная документация Django и Python, материалы по реляционным базам данных, статьи и учебные пособия по веб-разработке, а также практический опыт использования фитнес-центров в роли клиента. Сочетание технической документации и наблюдения за реальной работой фитнес-клуба помогло точнее определить состав сущностей и ролей в системе.

Научная новизна работы заключается в том, что для типовой задачи учета клиентов и расписания предложена компактная архитектура из шести приложений Django, разделенных по предметному принципу. Каждое приложение отвечает за одну часть предметной области и может развиваться отдельно от остальных. Такой подход облегчает сопровождение системы и позволяет постепенно усложнять отдельные блоки без переписывания всего проекта.

1. ОБЩАЯ ЧАСТЬ

1.1 Анализ объектной области

Фитнес-центр — это организация, где клиент получает спортивно-оздоровительные услуги. В клубе могут быть тренажерный зал, групповые занятия, персональные тренировки, силовые программы, кардио, йога, кроссфит и другие направления. Для нормальной работы такого места мало просто составить расписание. Нужно учитывать клиентов, тренеров, залы, абонементы, оплаты и записи на занятия.

Главные участники процесса — администратор, тренер и клиент. Администратор обычно отвечает за регистрацию клиентов, оформление абонементов, контроль оплат, расписание и общую организацию работы. Тренер проводит занятия, видит свой график и список тренировок. Клиент покупает абонемент, пользуется услугами фитнес-центра и записывается на занятия.

В объектной области можно выделить несколько основных процессов. Это регистрация пользователя, хранение личных данных, учет состояния здоровья и контактов для экстренной связи, работа со списком тренеров и специализаций, создание тарифных планов, оформление абонементов, фиксация платежей, планирование тренировок, назначение зала и запись клиента на занятие. Все эти процессы связаны между собой, поэтому их удобно хранить в одной системе.

При ручной работе быстро появляются типичные сложности. Данные о клиентах могут повторяться в разных местах. Срок действия абонемента приходится проверять вручную. Расписание становится трудно поддерживать актуальным, особенно если несколько тренеров работают в разных залах. Руководителю нужна сводная информация, а ее приходится собирать по частям.

Автоматизация помогает собрать эти данные в одной точке. Администратор может быстро посмотреть количество активных клиентов, список тренеров,

действующие абонементы, ближайшие занятия и последние платежи. Тренер видит расписание со временем, залом и названием тренировки. Клиент получает доступ к своим данным и не зависит от того, есть ли рядом администратор.

Особенность фитнес-центра в том, что система работает не только с организационными данными, но и с персональной информацией. В приложении есть данные клиентов и сотрудников, поэтому нужны авторизация, роли и аккуратное разграничение доступа. Пользователь должен видеть только те разделы, которые относятся к его роли.

Проект выбран в веб-формате. Это удобно для небольшой организации: приложение можно открыть через браузер, без установки отдельной программы на каждый компьютер. Администратор может работать с рабочего места, тренер — смотреть расписание со своего устройства, клиент — заходить в личный кабинет тогда, когда ему нужно.

Для фитнес-центра автоматизированная система становится рабочим инструментом, а не просто базой данных. Она связывает клиентов, тренеров, абонементы, платежи и расписание в одну понятную схему.

Дополнительно стоит отметить, что фитнес-центр работает в условиях постоянного притока новых клиентов и регулярных оплат. Если данные не собраны в одной системе, администратор каждый раз тратит время на поиск нужной записи, а ошибка в расписании может привести к недовольству нескольких посетителей сразу. Автоматизированная система снижает эту нагрузку: вся информация по клиенту, его абонементу и записям доступна в одном кабинете.

Еще одна особенность объектной области — необходимость периодически закрывать абонементы по истечении срока, переносить занятия и отмечать посещаемость. Все эти действия удобнее выполнять в системе, где такие операции уже встроены как стандартные функции, а не как разовые правки в таблице. Это

снижает количество ошибок и делает работу администратора более предсказуемой.

1.2 Анализ предметной области

Предметная область проекта включает те процессы, которые должны быть отражены в приложении. В системе фитнес-центра используются пользователи, клиентские профили, тренерские профили, специализации, тарифные планы, абонементы, платежи, залы, тренировки и записи на занятия.

Центральная сущность — пользователь. У пользователя есть учетная запись, логин, пароль, имя, фамилия, электронная почта, телефон, фотография и роль. Роль определяет, что именно человек может делать в системе. В проекте используются три роли: администратор, тренер и клиент. Это позволяет хранить общие данные в одной модели, а доступ к функциям разделять по назначению.

Профиль клиента содержит сведения, которые нужны для обслуживания посетителя: дату рождения, пол, адрес, контакт для экстренной связи, заметки о здоровье, дату регистрации и признак активности. Эти данные не стоит смешивать с обычной учетной записью, поэтому они вынесены в отдельную модель. Когда создается пользователь с ролью клиента, профиль формируется автоматически.

Профиль тренера хранит специализации, опыт работы, биографию, почасовую ставку, дату найма и статус активности. Специализации сделаны отдельной сущностью. Один тренер может вести несколько направлений, например силовые тренировки и кроссфит, а одно направление может быть закреплено за несколькими тренерами. Такая связь хорошо подходит для реальной работы фитнес-клуба.

Абонементы в проекте делятся на тарифные планы и конкретные абонементы клиентов. Тарифный план описывает название, тип, срок действия, цену, число тренировок и дополнительные условия. Конкретный абонемент связан с клиентом, имеет дату начала, дату окончания, статус, цену и количество

оставшихся посещений. Благодаря этому можно отдельно управлять шаблонами тарифов и уже оформленными абонементом.

Платежи связаны с абонементом. В платеже фиксируются сумма, дата, способ оплаты, статус и номер транзакции. Такой учет нужен администратору и руководителю, потому что финансовые данные должны быть понятными и проверяемыми.

Расписание строится через залы, тренировки и записи. Зал имеет название, вместимость, расположение и доступное оборудование. Тренировка содержит название, описание, тренера, зал, дату и время начала, продолжительность, максимальное количество участников, уровень сложности, статус и цену. Запись связывает клиента с тренировкой и хранит состояние записи: подтверждена, отменена, посещена или пропущена.

Предметная область показывает, что система должна быть связной. Нельзя рассматривать абонементы отдельно от платежей, тренировки отдельно от залов, а клиентов отдельно от записей. В проекте эти данные объединены через модели Django, что делает приложение понятным для разработки и дальнейшего расширения.

С точки зрения базы данных предметная область описывается набором сущностей и связей между ними. Каждая сущность становится таблицей, а связи реализуются через внешние ключи. Отдельно стоит подчеркнуть три типа связей: один к одному, один ко многим и многие ко многим. Связь один к одному используется между пользователем и его профилем клиента или тренера. Связь один ко многим — между клиентом и его абонементом, между абонементом и платежами, между тренером и его занятиями. Связь многие ко многим — между тренерами и специализациями, а также между клиентами и тренировками через таблицу записей.

Такое разделение помогает избежать дублирования данных. Например, имя и фамилия пользователя хранятся только в таблице учетных записей, а не в каждой записи на тренировку. Если клиент сменит фамилию, ее достаточно обновить в одном месте, и все остальные таблицы автоматически покажут новое значение через связи. Это типичный пример нормализации базы данных и одна из причин, по которым реляционная модель остается удобной для учета.

1.3 Описание средств и среды разработки

Для разработки выбран язык программирования Python. Он хорошо подходит для веб-приложений, потому что имеет понятный синтаксис, большое количество библиотек и активно используется вместе с фреймворком Django. Python позволяет быстро описывать модели данных, бизнес-логику и обработку пользовательских запросов.

Основой проекта стал Django. Это веб-фреймворк, который уже содержит многие нужные механизмы: маршрутизацию, работу с базой данных через ORM, систему шаблонов, авторизацию, административную панель, формы и защиту от типичных ошибок веб-разработки. Для дипломного проекта это удобно, потому что можно сосредоточиться на предметной области фитнес-центра, а не писать базовые части приложения с нуля.

В качестве базы данных используется SQLite. Для учебного проекта и небольшой системы это разумный вариант: база хранится в одном файле, не требует отдельного сервера и легко переносится. При дальнейшем развитии проекта базу можно заменить на PostgreSQL, не переписывая всю структуру приложения, потому что Django работает с данными через ORM.

Для интерфейса применяется Bootstrap. С его помощью страницы выглядят аккуратно и остаются удобными на разных размерах экрана. В проекте используются навигация, карточки, таблицы, формы и стандартные элементы

оформления. Это помогает быстрее сделать рабочий интерфейс без сложной ручной верстки.

В проекте также используется стандартная административная панель Django. Через нее можно управлять пользователями, клиентами, тренерами, абонементом, платежами, залами и тренировками. Админ-панель особенно полезна для администратора, потому что дает готовый инструмент для просмотра и редактирования данных.

Среда разработки включает структуру Django-проекта с отдельными приложениями. В проекте есть приложения `users`, `clients`, `trainers`, `subscriptions`, `schedule` и `reports`. Такой подход помогает разделить код по смыслу: пользователи находятся отдельно от абонементов, расписание отдельно от тренеров, а отчеты можно развивать позже.

Для контроля зависимостей используется файл `requirements.txt`. В нем перечислены библиотеки, нужные для запуска проекта. Такой файл позволяет быстро развернуть приложение на другом компьютере и получить такую же рабочую среду.

Выбранные средства подходят для задачи дипломной работы. Они не перегружают проект лишней сложностью, но дают все нужное для разработки веб-системы с базой данных, ролями пользователей и понятным интерфейсом.

Для работы с проектом используется редактор Visual Studio Code. Он бесплатный, имеет удобную подсветку синтаксиса для Python и HTML, поддерживает работу с виртуальными окружениями и системой контроля версий. Запуск тестовых команд и сервера разработки выполняется в интегрированном терминале редактора. Дополнительно используется браузер для проверки внешнего вида страниц и инструменты разработчика для отладки HTML и CSS.

Для управления исходным кодом применяется система контроля версий Git. Она помогает сохранять историю изменений, возвращаться к предыдущим

версиям файлов и видеть, какие именно правки были внесены в проект. Использование Git особенно полезно при долгой работе над дипломом, потому что позволяет не бояться экспериментировать с кодом и в любой момент откатиться к рабочему состоянию.

Запуск проекта выполняется встроенной командой `Django manage.py runserver`. Она поднимает локальный сервер разработки на адресе `127.0.0.1:8000`. Для применения структуры базы данных используются команды `makemigrations` и `migrate`. Команда `createsuperuser` создает учетную запись администратора, через которую можно войти в стандартную панель Django.

1.4 Обзор существующих вариантов программных продуктов

На российском рынке программных продуктов для фитнес-центров представлено несколько коммерческих решений. Ниже рассмотрены три наиболее популярных продукта и приведены скриншоты их интерфейсов.

1) 1С:Фитнес клуб. Российская конфигурация на платформе 1С:Предприятие 8. Поддерживает учёт клиентов, продажу абонементов, расписание занятий, кассовые операции и интеграцию с фитнес-оборудованием. Подходит крупным сетям клубов, но требует установленной платформы 1С и услуг специалиста по внедрению.

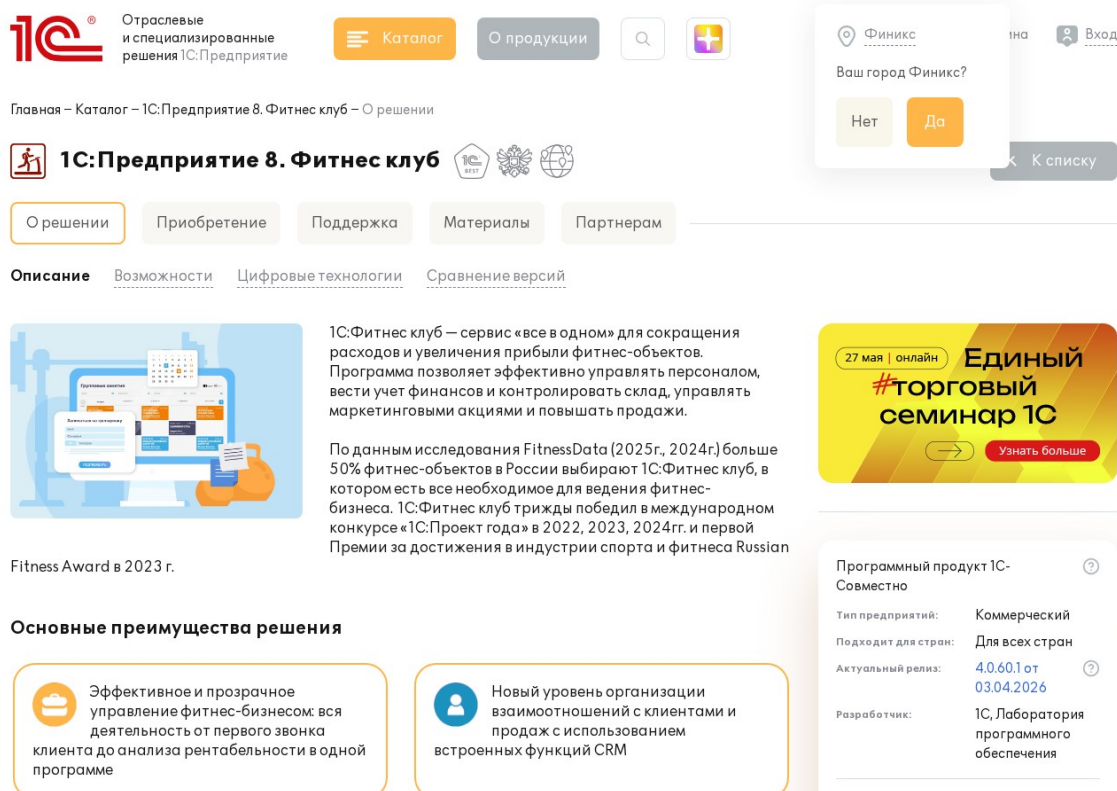


Рисунок 1 — Интерфейс системы «1С:Фитнес клуб»

2) Mobifitness. Облачная CRM-система для фитнес-клубов. Включает онлайн-расписание, личный кабинет клиента, мобильное приложение, оплату абонементов и пуш-уведомления. Распространяется по подписке, стоимость зависит от количества клиентов и сотрудников клуба.

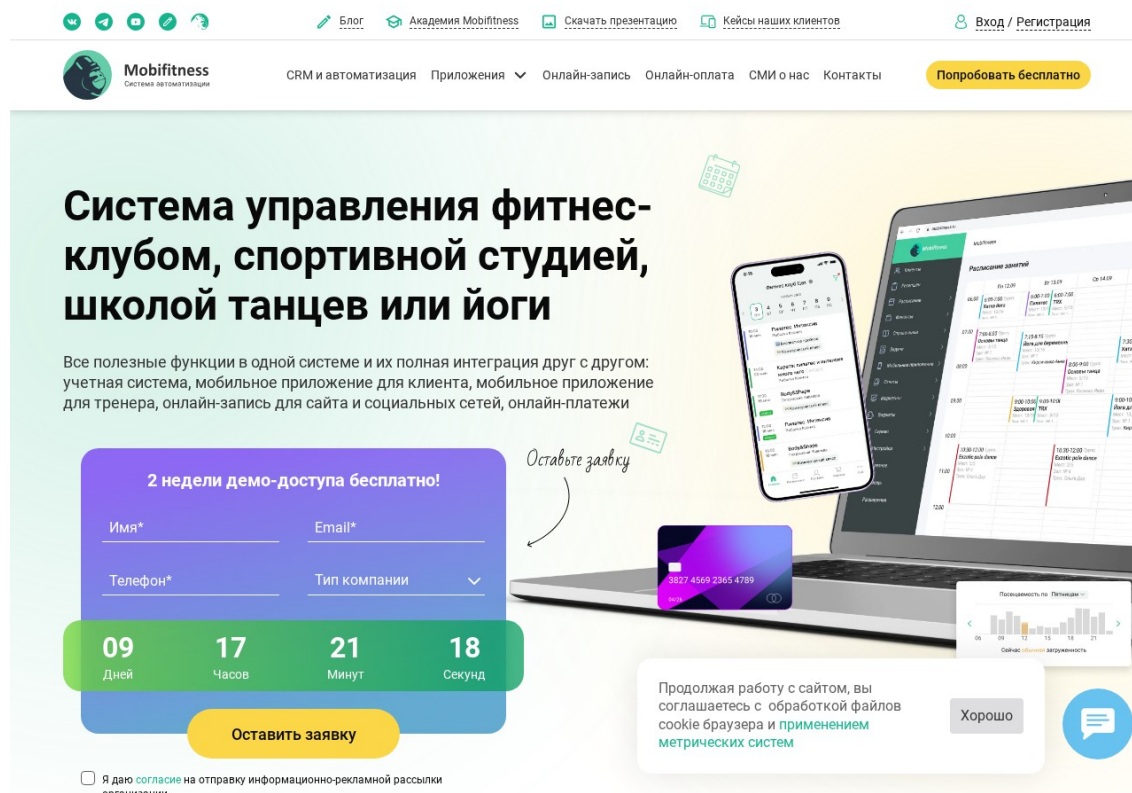


Рисунок 2 — Интерфейс системы «Mobifitness»

3) YCLIENTS. Универсальная CRM для сферы услуг, в том числе фитнес-центров. Реализует онлайн-запись клиентов, складской учёт, программу лояльности и аналитику. Среди особенностей — глубокая интеграция с социальными сетями и виджет онлайн-записи для сайта.

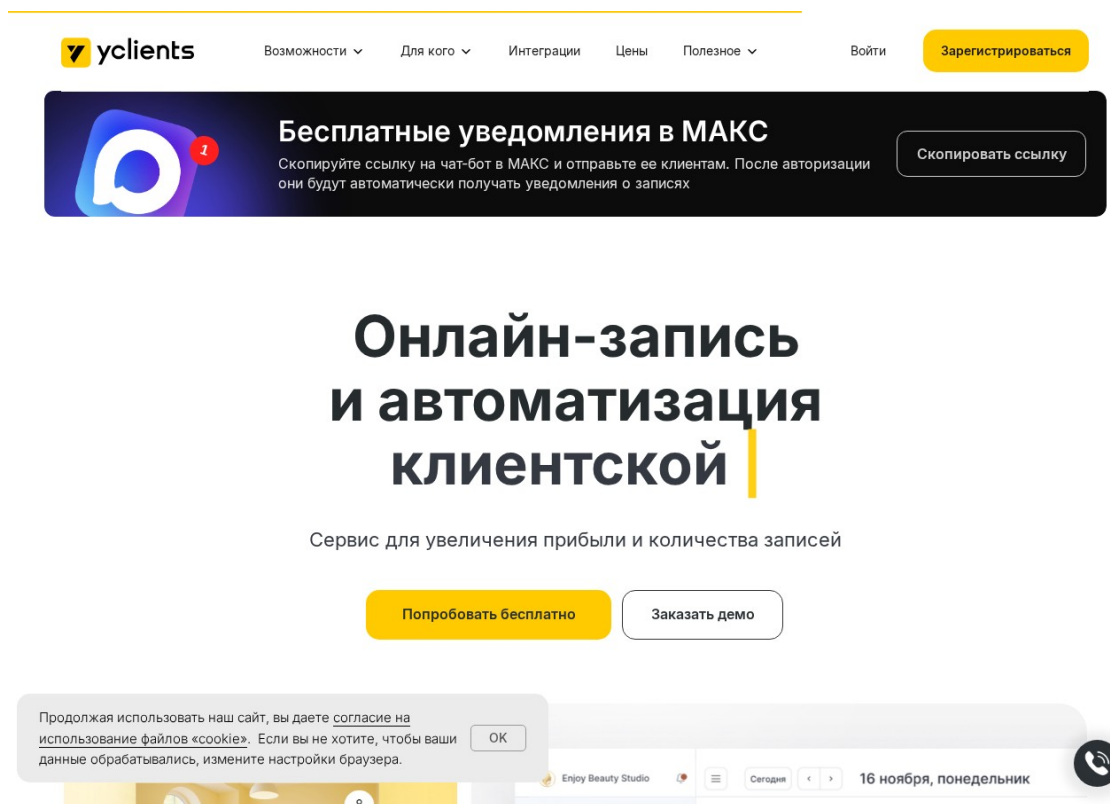


Рисунок 3 — Интерфейс системы «YCLIENTS»

Рассмотренные коммерческие системы избыточны по функциям и стоимости для небольшого фитнес-центра, а также требуют покупки лицензий или оплаты подписки. Это обосновывает разработку собственного учебного решения с базовым набором функций.

2. СПЕЦИАЛЬНАЯ ЧАСТЬ

2.1 Описание требований к информационной системе

Информационная система фитнес-центра должна решать задачи трех групп пользователей: администратора, тренера и клиента. Для каждого пользователя нужен свой личный кабинет и свой набор возможностей. Если человек вошел как клиент, он не должен видеть административные разделы. Если пользователь является тренером, ему нужен доступ к расписанию своих занятий, а не ко всем управленческим функциям.

Для входа в систему используется регистрация и авторизация. Пользователь создает учетную запись, указывает основные данные и получает роль. После входа система направляет его в нужный кабинет. Такой маршрут делает работу проще: пользователю не нужно искать свой раздел вручную.

Администратор должен видеть сводную панель. В ней отображаются активные клиенты, тренеры, абонементы, ближайшие тренировки и платежи. Также администратор может работать через Django Admin, где доступны основные модели проекта. Это дает возможность управлять данными без отдельной разработки большого административного интерфейса.

Тренер должен видеть свои занятия. Для него особенно важны дата, время, зал, название тренировки и количество записанных клиентов. В дипломной версии система делает упор на просмотр расписания, а дальнейшее развитие может добавить отметку посещаемости и связь с отчетами.

Клиенту нужен личный кабинет. В нем отображаются данные об активном абонементе, история абонементов и будущие записи на занятия. Такой кабинет помогает клиенту не уточнять каждую деталь у администратора.

К функциональным требованиям относятся регистрация, вход, выход, просмотр профиля, работа с ролями, учет клиентов и тренеров, создание

абонементов, фиксация платежей, формирование расписания и хранение записей на тренировки. К нефункциональным требованиям относятся понятный интерфейс, ограничение доступа по ролям, корректная работа форм, хранение данных в базе и возможность дальнейшего расширения.

Требования к системе можно описать через маршруты, модели и действия пользователя. Такой способ удобен для проверки: если маршрут открывается нужной роли, данные выводятся верно, а чужие разделы закрыты, значит базовая логика работает правильно.

Нефункциональные требования также важны. Система должна работать стабильно на типовом учебном оборудовании: обычный ноутбук с операционной системой Windows или Linux, установленный Python 3.10 и более новая версия, объем оперативной памяти от 4 ГБ. Время отклика на стандартные запросы должно быть в пределах одной-двух секунд, что для SQLite и небольшого объема данных достигается без специальных оптимизаций.

Дополнительно к нефункциональным требованиям относятся защищенность данных и удобство сопровождения. Пароли пользователей хранятся в базе только в виде хеша, а не в открытом виде. Это стандартное поведение Django: при сохранении пароля используется алгоритм PBKDF2 с солью. Такой подход защищает данные пользователей даже в случае несанкционированного доступа к базе данных.

Сводный список маршрутов и доступа к ним приведён в таблице 1.

Таблица 1 — Основные маршруты системы

Маршрут	Назначение	Доступ
/	Главная страница проекта	Все пользователи
/users/register/	Регистрация нового пользователя	Гость
/users/login/	Вход в систему	Гость
/users/logout/	Выход из системы	Авторизованный пользователь
/users/profile/	Просмотр профиля	Авторизованный пользователь
/users/dashboard/	Перенаправление в кабинет по	Авторизованный пользователь

Маршрут	Назначение	Доступ
	роли	
/users/admin-dashboard/	Кабинет администратора	Администратор
/users/trainer-dashboard/	Кабинет тренера	Тренер
/users/client-dashboard/	Кабинет клиента	Клиент
/admin/	Административная панель Django	Администратор

Перечень функциональных и нефункциональных требований приведён в таблице 2.

Таблица 2 — Основные требования к системе

Группа требований	Содержание требования
Авторизация	Пользователь должен входить в систему по логину и паролю
Регистрация	Новый пользователь должен создавать учетную запись через форму
Роли	Система должна разделять администратора, тренера и клиента
Профили	Для клиента и тренера должны храниться отдельные профильные данные
Абонементы	Система должна учитывать планы, сроки, статусы и посещения
Платежи	Нужно хранить сумму, дату, способ оплаты и статус платежа
Расписание	Нужно хранить тренировки, залы, время и тренеров
Записи	Клиент должен быть связан с выбранной тренировкой через запись
Интерфейс	Страницы должны быть понятными и удобными для пользователя
Безопасность	Пользователь не должен открывать разделы чужой роли

2.2 Функциональный сценарий

Функциональный сценарий показывает, как пользователь проходит через систему. Работа начинается с главной страницы. На ней пользователь видит общее описание фитнес-центра и может перейти к регистрации или входу. После авторизации система проверяет роль и отправляет человека в нужный кабинет.

Если в систему входит администратор, он попадает на панель со сводными данными. Там отображаются количество клиентов, тренеров, активных

абонементов и ближайших тренировок. Также администратор видит последние платежи. Для подробного редактирования данных он может перейти в административную панель Django.

Если входит тренер, система показывает его кабинет. В нем выводятся занятия, которые связаны именно с этим тренером. Это избавляет от лишней информации и помогает быстро понять свой график.

Если входит клиент, он видит свой кабинет. В нем отображается активный абонемент, если он есть, история абонементов и предстоящие записи на занятия. Такой сценарий делает личный кабинет полезным даже без участия администратора.

При создании пользователя система автоматически формирует профиль клиента или тренера, если роль это требует. Такой механизм реализован через сигналы Django. Это уменьшает количество ручных действий и помогает не забыть создать связанный профиль.

Функциональный сценарий можно описать как цепочку: регистрация, вход, определение роли, переход в нужный кабинет, просмотр данных, выход из системы. Для администратора добавляется управление данными через админ-панель, для тренера — просмотр личного расписания, для клиента — просмотр абонемента и записей.

Отдельно стоит описать сценарий ошибки. Если пользователь вводит неверный пароль, система не пускает его в кабинет и показывает сообщение об ошибке. Если пользователь пытается открыть страницу, для которой у него нет прав, происходит перенаправление на дашборд его роли или на страницу входа. Такая обработка нештатных ситуаций повышает доверие к системе и делает ее предсказуемой для конечного пользователя.

Полный перечень функций по ролям пользователей приведён в таблице 3.

Таблица 3 — Основные функции системы

Функция	Описание	Основной пользователь
Регистрация	Создание новой учетной записи	Клиент, тренер, администратор
Авторизация	Вход в систему по логину и паролю	Все роли
Ролевое перенаправление	Переход в кабинет по роли пользователя	Все роли
Просмотр профиля	Отображение личных данных пользователя	Все роли
Сводка администратора	Показ клиентов, тренеров, абонементов, занятий и платежей	Администратор
Расписание тренера	Показ тренировок конкретного тренера	Тренер
Кабинет клиента	Показ абонемента и записей клиента	Клиент
Управление моделями	Редактирование данных через Django Admin	Администратор

2.3 Проектирование структуры данных

Структура данных — одна из главных частей проекта. Если модели продуманы плохо, дальше будет трудно расширять систему и поддерживать связи между сущностями. В приложении используется объектная модель Django, где каждая сущность описана отдельным классом.

Модель User расширяет стандартного пользователя Django. В ней добавлены роль, телефон и фотография. Роль хранится как выбор из трех вариантов: admin, trainer и client. Такой вариант удобен, потому что вся авторизация остается внутри Django, а дополнительные данные помогают разделять пользователей по задачам.

Модель Client связана с User отношением один к одному. Она хранит дату рождения, пол, адрес, контакт для экстренной связи, медицинские заметки, дату регистрации и активность. Эти данные нужны для работы с посетителями фитнес-центра.

Модель Trainer тоже связана с User отношением один к одному. В ней есть специализации, опыт работы, биография, почасовая ставка, дата найма и активность. Связь со специализациями сделана через many-to-many, потому что тренер может вести несколько направлений.

Блок абонементов состоит из SubscriptionPlan, Subscription и Payment. SubscriptionPlan описывает тариф: название, тип, срок, цену, количество занятий и условия. Subscription показывает конкретный абонемент клиента. Payment фиксирует оплату по абонементу.

Блок расписания состоит из GymRoom, TrainingClass и Enrollment. GymRoom хранит данные о зале. TrainingClass описывает занятие: тренера, зал, дату, время, длительность, вместимость, уровень и статус. Enrollment связывает клиента с занятием и хранит статус записи.

Такая структура хорошо отражает работу фитнес-центра. Пользователь связан с ролью, роль связана с профилем, клиент связан с абонементом и записями, тренер связан с тренировками, а тренировки связаны с залами. Данные не хранятся кучей в одной таблице, а разделены по смыслу.

Структура базы данных в Django описывается на языке Python в виде моделей. После каждого изменения моделей запускается команда makemigrations, которая создает файл миграции, а затем команда migrate, которая применяет изменения к базе. Такой подход называется миграциями базы данных и позволяет переносить структуру таблиц между разными копиями проекта без ручного написания SQL-скриптов.

Дополнительно при проектировании учитывались принципы нормализации. Каждая таблица отвечает за один тип данных, каждое поле хранит атомарное значение, а повторяющаяся информация выносится в отдельные таблицы и связывается через внешние ключи. Это позволяет избежать аномалий при добавлении, изменении и удалении данных, что особенно важно для финансовых записей и абонементов.

В проекте также предусмотрено приложение reports. Пока оно не содержит полноценной логики, но его наличие показывает направление для развития. В

будущем туда можно добавить отчеты по посещаемости, оплатам, загрузке залов и активности клиентов.

Список основных сущностей базы данных с указанием полей приведён в таблице 4.

Таблица 4 — Основные сущности базы данных

Сущность	Назначение	Основные поля
User	Учетная запись пользователя	username, password, email, phone, role, photo
Client	Профиль клиента	user, birth_date, gender, address, health_notes, is_active
Trainer	Профиль тренера	user, specializations, experience_years, bio, hourly_rate
Specialization	Направление работы тренера	name, description
SubscriptionPlan	Тарифный план	name, plan_type, duration_days, price, sessions_count
Subscription	Абонемент клиента	client, plan, start_date, end_date, status, remaining_sessions
Payment	Оплата абонемента	subscription, amount, payment_date, method, status
GymRoom	Тренировочный зал	name, capacity, location, equipment
TrainingClass	Занятие в расписании	title, trainer, room, start_datetime, duration_minutes, status
Enrollment	Запись клиента на занятие	client, training_class, enrolled_at, status

2.4 Описание ролей и прав доступа

Ролевая модель нужна для того, чтобы разные пользователи не работали в одной и той же зоне системы. В фитнес-центре администратор, тренер и клиент решают разные задачи, поэтому доступ должен отличаться.

Администратор имеет самый широкий доступ. Он может просматривать сводную информацию, работать с данными через административную панель, добавлять и редактировать клиентов, тренеров, абонементы, платежи, залы и занятия. В учебной версии основная управленческая работа выполняется через Django Admin.

Тренер получает доступ к своему кабинету. Его задача — видеть тренировки, которые закреплены за ним. В этом разделе ему не нужны финансовые данные всех клиентов или управление тарифами. Ограничение доступа делает интерфейс проще и снижает риск случайного изменения чужих данных.

Клиент видит личный кабинет. Там отображаются его абонементы и записи. Клиент не должен видеть данные других пользователей, список всех платежей и административные настройки.

В проекте доступ проверяется через декораторы. Для каждого закрытого раздела можно указать, какая роль имеет право его открыть. Если пользователь не авторизован, система отправляет его на вход. Если роль не подходит, доступ к странице ограничивается.

Такой подход понятен и легко расширяется. Если в будущем появится новая роль, например менеджер или бухгалтер, для нее можно добавить отдельный набор прав и кабинет.

Разграничение доступа выполнено на нескольких уровнях. На уровне базы данных каждый объект связан с конкретным пользователем через внешний ключ. На уровне представлений используется проверка декораторами `login_required` и собственные проверки роли. На уровне шаблонов нужные кнопки и ссылки отображаются только тем пользователям, у которых есть права на эти действия. Сочетание трех уровней снижает риск ошибочного доступа к чужим данным.

Сводное описание ролей пользователей и их возможностей приведено в таблице 5.

Таблица 5 — Роли пользователей

Роль	Основные возможности	Ограничения
Администратор	Сводка, управление данными, работа через Django Admin	Не предназначен для клиентского кабинета
Тренер	Просмотр личного расписания и своих занятий	Нет доступа к управлению платежами и тарифами

Роль	Основные возможности	Ограничения
Клиент	Просмотр абонеента, истории и записей	Нет доступа к административным данным
Гость	Просмотр главной страницы, регистрация и вход	Нет доступа к личным кабинетам

2.5 Пользовательские сценарии

Пользовательские сценарии помогают проверить, удобно ли работать с системой. Они описывают не только функции, но и обычный путь человека внутри приложения.

Первый сценарий — регистрация клиента. Пользователь открывает сайт, переходит к форме регистрации, вводит имя, логин, пароль, электронную почту и другие данные. После создания учетной записи система сохраняет пользователя и формирует профиль. Затем пользователь может войти и попасть в личный кабинет.

Второй сценарий — работа администратора. Администратор входит в систему, попадает на свою панель и видит общие показатели. Если нужно добавить новый тариф, тренировку или платеж, он открывает административную панель и редактирует данные в нужном разделе.

Третий сценарий — работа тренера. Тренер авторизуется и попадает в личный кабинет. На странице он видит занятия, которые назначены ему. Для него важны дата, время, название тренировки и зал. Такой сценарий помогает быстро узнать свой график без лишних разделов.

Четвертый сценарий — работа клиента. Клиент входит в систему, открывает кабинет и смотрит активный абонемент. Если абонементов несколько, он может видеть историю. Также клиент видит будущие записи на тренировки.

Пятый сценарий — выход из системы. После завершения работы пользователь нажимает выход, и система закрывает сессию. Это особенно нужно, если приложение используется на общем компьютере администратора или ресепшена.

Сценарии показывают, что приложение строится вокруг простых действий. Пользователь входит, получает свой раздел и работает с теми данными, которые относятся к его роли.

Помимо основных сценариев в системе предусмотрены вспомогательные операции. Это, например, переход к собственному профилю, изменение основных данных пользователя, обновление состояния абонента после его покупки. Эти операции не являются центральными для дипломной работы, но без них приложение выглядит неполным, поэтому они также входят в проверяемый набор сценариев.

Основные пользовательские сценарии работы с системой приведены в таблице 6.

Таблица 6 — Пользовательские сценарии

Сценарий	Шаги	Ожидаемый результат
Регистрация клиента	Открыть регистрацию, заполнить форму, отправить данные	Создан пользователь и профиль клиента
Вход администратора	Ввести логин и пароль администратора	Открыта административная панель со сводкой
Просмотр расписания тренера	Войти как тренер и открыть кабинет	Показаны занятия выбранного тренера
Просмотр кабинета клиента	Войти как клиент и открыть кабинет	Показан абонемент и записи клиента
Выход из системы	Нажать кнопку выхода	Сессия завершена, пользователь выходит из аккаунта
Попытка открыть чужой раздел	Клиент открывает адрес кабинета администратора	Доступ ограничен

2.6 Описание интерфейсов

Интерфейс проекта сделан через HTML-шаблоны Django и Bootstrap. Главная задача интерфейса — не перегружать пользователя. Страницы должны быть простыми: заголовок, навигация, основные данные и понятные кнопки.

Базовый шаблон `base.html` задает общую структуру страниц. В нем размещается навигация, подключаются стили Bootstrap и определяется место для

содержимого конкретной страницы. Благодаря этому все страницы выглядят в одном стиле.

Главная страница `home.html` знакомит пользователя с проектом. Она нужна для входа в систему и перехода к регистрации. На этой странице можно разместить краткое описание фитнес-центра и основные преимущества использования личного кабинета.

Кабинет администратора показывает сводные показатели. Для администратора важно видеть цифры и списки без лишнего поиска: сколько активных клиентов, сколько тренеров, какие абонементы действуют, какие тренировки скоро начнутся и какие платежи были последними.

Кабинет тренера построен вокруг расписания. В нем выводятся занятия, связанные с текущим тренером. Такой интерфейс не должен быть перегружен финансовыми или административными данными.

Кабинет клиента показывает личную информацию по абонементам и записям. Клиенту важно быстро понять, действует ли абонемент, сколько осталось посещений и есть ли ближайшие занятия.

Формы регистрации и профиля сделаны через формы Django. Это удобно, потому что фреймворк помогает проверять данные и выводить ошибки рядом с полями. Пользователь сразу видит, что нужно исправить.

Административная панель Django используется как рабочий интерфейс для управления данными. В ней настроены отображение списков, поиск, фильтры и поля для редактирования. Для дипломного проекта это хороший способ показать полноценную работу с моделями без создания отдельного большого модуля управления.

Цветовая схема интерфейса выполнена в темной теме. Основной фон страниц — темно-синий, акцентный цвет — оранжевый, дополнительные цвета — красно-оранжевый и розовый. Темная тема снижает нагрузку на глаза при длительной работе с системой, что особенно полезно для администратора. Все

цвета вынесены в отдельный CSS-файл, что позволяет легко изменить оформление без правки шаблонов.

Сообщения системы оформлены через стандартный механизм messages в Django. После успешной операции, например после сохранения профиля или создания абонемента, на следующей странице появляется зеленое уведомление. При ошибке выводится сообщение красного цвета. Это помогает пользователю получить обратную связь и понять, что действие прошло успешно.

Перечень интерфейсов системы и их назначение приведены в таблице 7.

Таблица 7 — Интерфейсы системы

Интерфейс	Назначение	Пользователь
Главная страница	Описание проекта, переход к входу и регистрации	Гость
Форма регистрации	Создание учетной записи	Гость
Форма входа	Авторизация пользователя	Все роли
Профиль	Просмотр личных данных	Авторизованный пользователь
Кабинет администратора	Сводная информация по фитнес-центру	Администратор
Кабинет тренера	Просмотр занятий тренера	Тренер
Кабинет клиента	Просмотр абонементов и записей	Клиент
Django Admin	Управление моделями и данными	Администратор

2.7 Тестирование и отладка

Тестирование нужно для того, чтобы убедиться: приложение не просто открывается, а правильно выполняет основные действия. В дипломном проекте проверяются регистрация, вход, перенаправление по ролям, доступ к кабинетам, отображение данных и работа административной панели.

Первым проверяется запуск проекта. После установки зависимостей нужно применить миграции и запустить сервер разработки Django. Если главная страница открывается без ошибок, можно переходить к проверке пользовательских сценариев.

Дальше проверяется регистрация. Нужно создать пользователя, убедиться, что форма принимает корректные данные и показывает ошибки при неправильном вводе. После регистрации проверяется, появился ли связанный профиль клиента или тренера.

Отдельно проверяется авторизация. Пользователь должен входить с правильным логином и паролем. При неверных данных система не должна открывать личный кабинет. После входа пользователь должен попадать в раздел своей роли.

Самая важная часть проверки — ролевой доступ. Клиент не должен открывать кабинет администратора. Тренер не должен получать доступ к разделам управления платежами и тарифами. Администратор должен открывать свою панель и административный интерфейс.

Проверяется и отображение данных. В кабинете администратора должны выводиться сводные показатели. В кабинете тренера — его тренировки. В кабинете клиента — абонементы и записи. Если данные в базе изменяются через админ-панель, это должно отражаться на страницах.

При отладке могут появляться ошибки маршрутов, неправильные связи между моделями, отсутствие профиля или неверная фильтрация данных. Такие ошибки исправляются через проверку моделей, представлений, шаблонов и декораторов доступа.

Для учебного проекта достаточно функционального тестирования основных сценариев. В дальнейшем можно добавить автоматические тесты Django для моделей, форм, представлений и проверки прав доступа.

Результаты ручного тестирования записываются в журнал тест-кейсов. В журнале указываются номер проверки, ее описание, ожидаемый результат, фактический результат и заключение о прохождении. Если фактический результат отличается от ожидаемого, ошибка фиксируется и устраняется. После исправления тест повторяется и его статус обновляется. Такой подход помогает не

пропустить дефекты и подтверждает, что система действительно работает по заявленным сценариям.

В дополнение к функциональным проверкам выполняется проверка корректности отображения страниц в разных размерах окна браузера. Благодаря Bootstrap большинство страниц адаптируется автоматически: навигация сворачивается в кнопку-меню на узких экранах, таблицы получают горизонтальную прокрутку. Это особенно важно, если администратор работает не только с настольного компьютера, но и с планшета.

Журнал ручного тестирования с проверками и ожидаемыми результатами приведён в таблице 8.

Таблица 8 — Тест-кейсы

Проверка	Действие	Ожидаемый результат
Запуск проекта	Выполнить миграции и запустить сервер	Главная страница открывается
Регистрация	Создать нового пользователя через форму	Пользователь сохранен в базе
Авторизация	Войти с правильными данными	Открывается личный кабинет
Неверный вход	Ввести неправильный пароль	Доступ не предоставлен
Роль администратора	Открыть кабинет администратора	Показана сводная информация
Роль тренера	Открыть кабинет тренера	Показано расписание тренера
Роль клиента	Открыть кабинет клиента	Показаны абонементы и записи
Защита разделов	Открыть раздел чужой роли	Система ограничивает доступ
Админ-панель	Открыть /admin/ под администратором	Доступны модели проекта

ЗАКЛЮЧЕНИЕ

В ходе дипломной работы была разработана автоматизированная информационная система для фитнес-центра. Проект решает задачу централизованного хранения данных о пользователях, клиентах, тренерах, абонементах, платежах, залах, тренировках и записях на занятия.

В общей части была рассмотрена деятельность фитнес-центра и показано, почему ручной учет создает проблемы. Были описаны основные пользователи системы, процессы предметной области, инструменты разработки и существующие аналоги. Это помогло определить, какие функции действительно нужны приложению.

В специальной части были описаны требования, функциональные сценарии, структура данных, роли пользователей, интерфейсы и тестирование. Особое внимание уделено разделению доступа. Администратор, тренер и клиент получают разные кабинеты и разные возможности, что соответствует реальной работе фитнес-центра.

Практическая часть реализована на Python и Django. В проекте используются модели данных, маршруты, представления, шаблоны, формы, сигналы и административная панель. SQLite применяется для хранения данных, Bootstrap — для оформления страниц.

Полученное приложение можно использовать как основу для дальнейшего развития. В него можно добавить онлайн-запись клиента на занятия, отметку посещаемости, уведомления, отчеты по оплатам, аналитику загрузки залов, интеграцию с внешними платежными сервисами и более подробный клиентский интерфейс.

Поставленная цель достигнута: система спроектирована и описана как веб-приложение для автоматизации работы фитнес-центра. Она показывает, как

можно связать учет клиентов, тренеров, абонементов, платежей и расписания в одном проекте.

В ходе работы над проектом был получен опыт разработки веб-приложений на Python и Django, проектирования реляционных баз данных, оформления пользовательских интерфейсов на базе Bootstrap, организации ролевой модели и подготовки технической документации. Этот опыт применим в дальнейшей профессиональной деятельности при разработке информационных систем для других предметных областей.

Дальнейшее развитие проекта может идти в нескольких направлениях. Первое — расширение клиентского интерфейса: добавление самостоятельной записи клиента на тренировки, личного календаря занятий, истории посещений. Второе — добавление аналитики: отчетов по выручке, посещаемости и загрузке тренеров. Третье — интеграция с внешними сервисами: онлайн-оплата, отправка уведомлений через мессенджеры, экспорт данных в бухгалтерские системы. Каждое из этих направлений требует отдельной проработки, но архитектура текущего проекта позволяет добавлять такие функции постепенно.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Django Software Foundation. Документация Django [Электронный ресурс]. — URL: <https://docs.djangoproject.com/> (дата обращения: 15.04.2026).
2. Python Software Foundation. Документация языка Python 3 [Электронный ресурс]. — URL: <https://docs.python.org/3/> (дата обращения: 15.04.2026).
3. Bootstrap. Документация Bootstrap [Электронный ресурс]. — URL: <https://getbootstrap.com/docs/> (дата обращения: 15.04.2026).
4. SQLite. Документация SQLite [Электронный ресурс]. — URL: <https://www.sqlite.org/docs.html> (дата обращения: 15.04.2026).
5. MDN Web Docs. Документация по HTML [Электронный ресурс]. — URL: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 15.04.2026).
6. MDN Web Docs. Обзор протокола HTTP [Электронный ресурс]. — URL: <https://developer.mozilla.org/ru/docs/Web/HTTP> (дата обращения: 15.04.2026).
7. Django Software Foundation. Система аутентификации Django [Электронный ресурс]. — URL: <https://docs.djangoproject.com/en/stable/topics/auth/> (дата обращения: 15.04.2026).
8. Django Software Foundation. Модели Django [Электронный ресурс]. — URL: <https://docs.djangoproject.com/en/stable/topics/db/models/> (дата обращения: 15.04.2026).
9. Django Software Foundation. Административная панель Django [Электронный ресурс]. — URL: <https://docs.djangoproject.com/en/stable/ref/contrib/admin/> (дата обращения: 15.04.2026).
10. Git. Документация Git [Электронный ресурс]. — URL: <https://git-scm.com/doc> (дата обращения: 15.04.2026).
11. Гринберг М. Разработка веб-приложений с использованием Flask на языке Python. — М.: ДМК Пресс, 2014. — 272 с.

12. Любанович Б. Простой Python. Современный стиль программирования. — СПб.: Питер, 2021. — 480 с.
13. Карпова Т. С. Базы данных: модели, разработка, реализация. — СПб.: Питер, 2001. — 304 с.
14. Дейт К. Дж. Введение в системы баз данных. — М.: Вильямс, 2005. — 1328 с.
15. Соммервилл И. Инженерия программного обеспечения. — М.: Вильямс, 2017. — 1032 с.
16. Куликов С. С. Тестирование программного обеспечения. Базовый курс. — Минск: Четыре четверти, 2017. — 312 с.
17. ГОСТ 34.602-2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. — М.: Стандартинформ, 2020. — 12 с.
18. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» [Электронный ресурс]. — URL: https://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 15.04.2026).

ПРИЛОЖЕНИЕ А. ЛИСТИНГИ ИСХОДНОГО КОДА

В приложении приведены ключевые фрагменты исходного кода разработанной системы. Полный код проекта размещён в репозитории на GitHub. Листинги выполнены шрифтом Times New Roman 11 пт с межстрочным интервалом 1,0.

Листинг — файл users/models.py. Модель пользователя с указанием роли (администратор, тренер, клиент).

```
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    ROLE_CHOICES = [
        ('admin', 'Администратор'),
        ('trainer', 'Тренер'),
        ('client', 'Клиент'),
    ]

    role = models.CharField(
        max_length=20,
        choices=ROLE_CHOICES,
        default='client',
        verbose_name='Роль'
    )
    phone = models.CharField(
        max_length=15,
        blank=True,
        null=True,
        verbose_name='Телефон'
    )
    photo = models.ImageField(
        upload_to='users/photos/',
        blank=True,
        null=True,
        verbose_name='Фото'
    )

    class Meta:
        verbose_name = 'Пользователь'
        verbose_name_plural = 'Пользователи'

    def __str__(self):
        return f'{self.get_full_name()} ({self.get_role_display()})'
```

```

@property
def is_admin(self):
    return self.role == 'admin'

@property
def is_trainer(self):
    return self.role == 'trainer'

@property
def is_client(self):
    return self.role == 'client'

```

Листинг — файл users/forms.py. Формы регистрации клиента и редактирования профиля.

```

from django import forms
from django.contrib.auth.forms import UserChangeForm, UserCreationForm

from clients.models import Client

from .models import User

class ClientRegistrationForm(UserCreationForm):
    first_name = forms.CharField(label='Имя', max_length=150,
required=True)
    last_name = forms.CharField(label='Фамилия', max_length=150,
required=True)
    email = forms.EmailField(label='Email', required=True)
    phone = forms.CharField(label='Телефон', max_length=15, required=False)

    class Meta:
        model = User
        fields = (
            'username', 'first_name', 'last_name', 'email', 'phone',
            'password1', 'password2',
        )

    def save(self, commit=True):
        user = super().save(commit=False)
        user.role = 'client'
        user.first_name = self.cleaned_data['first_name']
        user.last_name = self.cleaned_data['last_name']
        user.email = self.cleaned_data['email']
        user.phone = self.cleaned_data.get('phone') or ''
        if commit:
            user.save()
            Client.objects.get_or_create(user=user)

```

```
return user
```

```
class UserProfileForm(UserChangeForm):  
    password = None  
  
    class Meta:  
        model = User  
        fields = ('first_name', 'last_name', 'email', 'phone', 'photo')
```

Листинг — файл users/decorators.py. Декораторы для проверки роли пользователя при доступе к разделам.

```
from functools import wraps  
  
from django.contrib.auth.decorators import login_required  
from django.core.exceptions import PermissionDenied  
  
def _check_role(view_func, allowed_roles):  
    @wraps(view_func)  
    @login_required  
    def wrapper(request, *args, **kwargs):  
        user = request.user  
        if user.is_superuser or user.role in allowed_roles:  
            return view_func(request, *args, **kwargs)  
        raise PermissionDenied('Недостаточно прав.')  
    return wrapper  
  
def admin_required(view_func):  
    return _check_role(view_func, ['admin'])  
  
def trainer_required(view_func):  
    return _check_role(view_func, ['trainer'])  
  
def client_required(view_func):  
    return _check_role(view_func, ['client'])  
  
def staff_required(view_func):  
    return _check_role(view_func, ['admin', 'trainer'])
```

Листинг — файл users/views.py. Представления для регистрации, входа, дашбордов и профиля.

```

from django.contrib import messages
from django.contrib.auth import login
from django.contrib.auth.decorators import login_required
from django.shortcuts import redirect, render
from django.urls import reverse
from django.utils import timezone

from clients.models import Client
from schedule.models import Enrollment, TrainingClass
from subscriptions.models import Payment, Subscription
from trainers.models import Trainer

from .forms import ClientRegistrationForm, UserProfileForm

def home(request):
    if request.user.is_authenticated:
        return redirect('users:dashboard')
    return render(request, 'home.html')

def register(request):
    if request.user.is_authenticated:
        return redirect('users:dashboard')

    if request.method == 'POST':
        form = ClientRegistrationForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, 'Регистрация прошла успешно! Добро пожаловать.')
            return redirect('users:dashboard')
        else:
            form = ClientRegistrationForm()
            return render(request, 'users/register.html', {'form': form})

@login_required
def dashboard(request):
    user = request.user
    if user.is_superuser or user.role == 'admin':
        return admin_dashboard(request)
    if user.role == 'trainer':
        return trainer_dashboard(request)
    return client_dashboard(request)

def admin_dashboard(request):
    today = timezone.now().date()

```

```

now = timezone.now()
context = {
    'clients_count': Client.objects.filter(is_active=True).count(),
    'trainers_count': Trainer.objects.filter(is_active=True).count(),
    'active_subscriptions': Subscription.objects.filter(
        status='active', end_date__gte=today,
    ).count(),
    'upcoming_classes': TrainingClass.objects.filter(
        start_time__gte=now, status='scheduled',
    ).order_by('start_time')[:5],
    'recent_payments': Payment.objects.select_related(
        'subscription__client__user',
    ).order_by('-paid_at')[:5],
}
return render(request, 'users/dashboard_admin.html', context)

```

```

def trainer_dashboard(request):
    trainer = getattr(request.user, 'trainer_profile', None)
    classes = []
    if trainer:
        classes = TrainingClass.objects.filter(
            trainer=trainer, start_time__gte=timezone.now(),
        ).order_by('start_time')[:10]
    return render(request, 'users/dashboard_trainer.html', {
        'trainer': trainer,
        'upcoming_classes': classes,
    })

```

```

def client_dashboard(request):
    client = getattr(request.user, 'client_profile', None)
    today = timezone.now().date()
    subscriptions = []
    enrollments = []
    active_subscription = None
    if client:
        subscriptions =
list(Subscription.objects.select_related('plan').filter(
    client=client,
).order_by('-start_date')[:5])
        enrollments = Enrollment.objects.select_related(
            'training_class__trainer__user', 'training_class__room',
        ).filter(
            client=client, training_class__start_time__gte=timezone.now(),
        ).order_by('training_class__start_time')[:10]
        for s in subscriptions:
            if s.status == 'active' and s.end_date >= today:
                active_subscription = s
                break
    return render(request, 'users/dashboard_client.html', {
        'client': client,

```

```

        'subscriptions': subscriptions,
        'active_subscription': active_subscription,
        'enrollments': enrollments,
    })

```

```

@login_required
def profile(request):
    if request.method == 'POST':
        form = UserProfileForm(request.POST, request.FILES,
instance=request.user)
        if form.is_valid():
            form.save()
            messages.success(request, 'Профиль обновлён.')
            return redirect(reverse('users:profile'))
    else:
        form = UserProfileForm(instance=request.user)
    return render(request, 'users/profile.html', {'form': form})

```

Листинг — файл `schedule/models.py`. Модели залов, тренировок и записей

КЛИЕНТОВ НА ЗАНЯТИЯ.

```

from django.db import models
from clients.models import Client
from trainers.models import Trainer

class GymRoom(models.Model):
    name = models.CharField(
        max_length=100,
        verbose_name='Название зала'
    )
    capacity = models.PositiveIntegerField(
        verbose_name='Вместимость'
    )
    description = models.TextField(
        blank=True,
        null=True,
        verbose_name='Описание'
    )

    class Meta:
        verbose_name = 'Зал'
        verbose_name_plural = 'Залы'

    def __str__(self):
        return f'{self.name} (вместимость: {self.capacity})'

class TrainingClass(models.Model):
    STATUS_CHOICES = [

```

```

        ('scheduled', 'Запланировано'),
        ('in_progress', 'Идёт'),
        ('completed', 'Завершено'),
        ('cancelled', 'Отменено'),
    ]

    name = models.CharField(
        max_length=200,
        verbose_name='Название тренировки'
    )
    trainer = models.ForeignKey(
        Trainer,
        on_delete=models.SET_NULL,
        null=True,
        related_name='classes',
        verbose_name='Тренер'
    )
    room = models.ForeignKey(
        GymRoom,
        on_delete=models.SET_NULL,
        null=True,
        related_name='classes',
        verbose_name='Зал'
    )
    start_time = models.DateTimeField(
        verbose_name='Начало'
    )
    end_time = models.DateTimeField(
        verbose_name='Конец'
    )
    max_participants = models.PositiveIntegerField(
        verbose_name='Макс. участников'
    )
    status = models.CharField(
        max_length=20,
        choices=STATUS_CHOICES,
        default='scheduled',
        verbose_name='Статус'
    )
    description = models.TextField(
        blank=True,
        null=True,
        verbose_name='Описание'
    )

    class Meta:
        verbose_name = 'Тренировка'
        verbose_name_plural = 'Тренировки'
        ordering = ['start_time']

    def __str__(self):
        return f'{self.name} - {self.start_time.strftime("%d.%m.%Y %H:%M")}'

```



```

@property
def available_spots(self):
    return self.max_participants - self.enrollments.count()

@property
def is_full(self):
    return self.available_spots <= 0

@property
def duration_minutes(self):
    delta = self.end_time - self.start_time
    return int(delta.total_seconds() / 60)

class Enrollment(models.Model):
    STATUS_CHOICES = [
        ('enrolled', 'Записан'),
        ('attended', 'Посетил'),
        ('missed', 'Пропустил'),
        ('cancelled', 'Отменил'),
    ]

    client = models.ForeignKey(
        Client,
        on_delete=models.CASCADE,
        related_name='enrollments',
        verbose_name='Клиент'
    )
    training_class = models.ForeignKey(
        TrainingClass,
        on_delete=models.CASCADE,
        related_name='enrollments',
        verbose_name='Тренировка'
    )
    enrolled_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name='Дата записи'
    )
    status = models.CharField(
        max_length=20,
        choices=STATUS_CHOICES,
        default='enrolled',
        verbose_name='Статус'
    )

    class Meta:
        verbose_name = 'Запись на тренировку'
        verbose_name_plural = 'Записи на тренировки'
        unique_together = ['client', 'training_class']

```

```
def __str__(self):  
    return f'{self.client} → {self.training_class}'  
# Create your models here.
```

ПРИЛОЖЕНИЕ Б. ИНТЕРФЕЙСЫ СИСТЕМЫ

В приложении приведены скриншоты основных экранов разработанной системы. Скриншоты сделаны в браузере на локально запущенном экземпляре приложения.

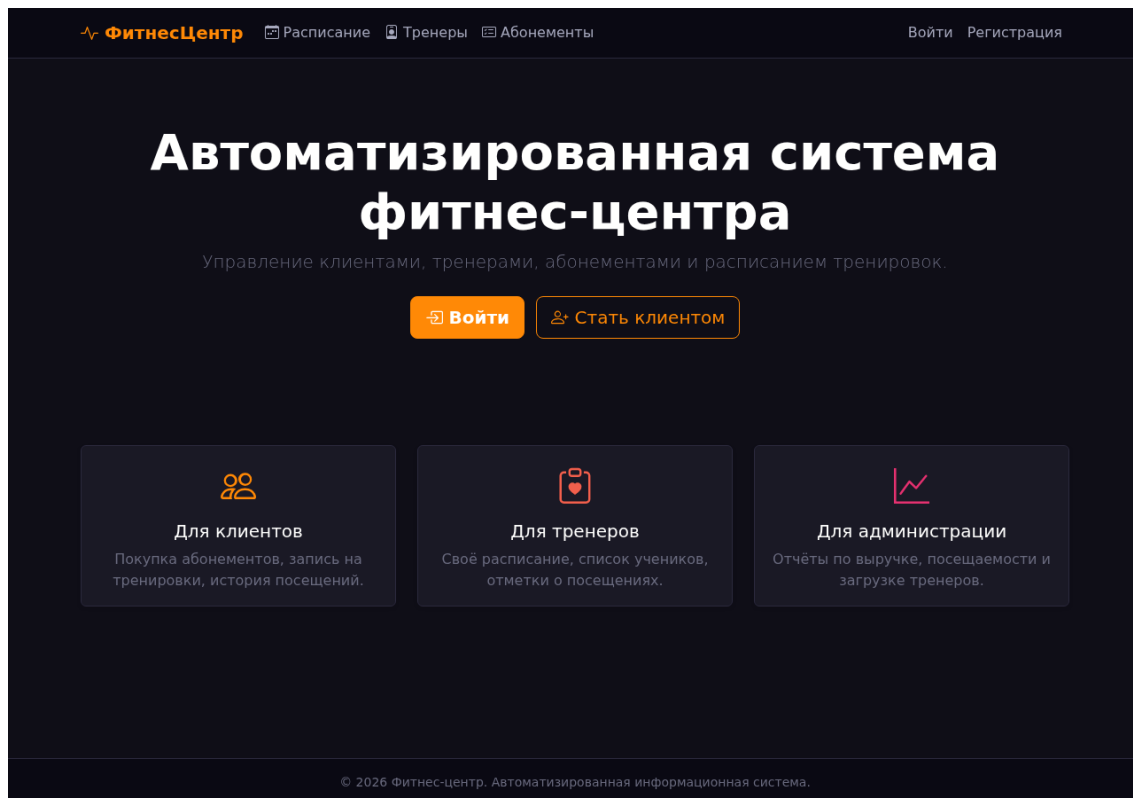


Рисунок 4 — Главная страница приложения

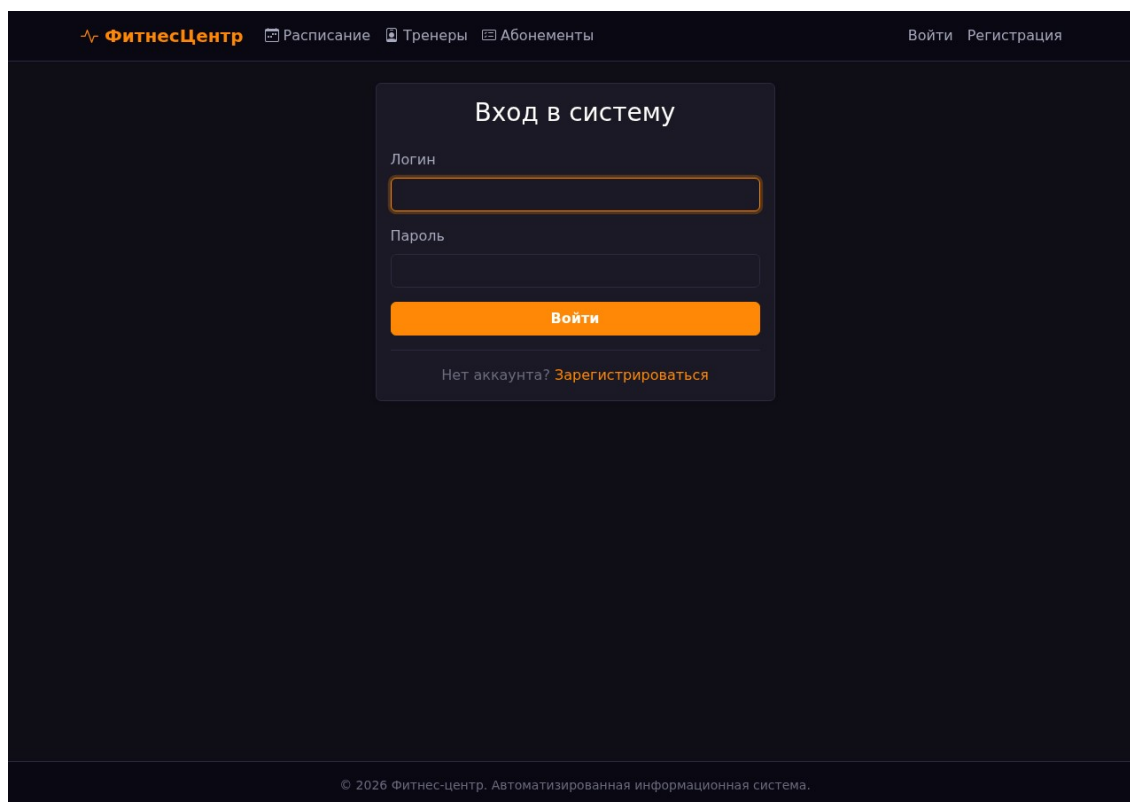


Рисунок 5 — Страница входа в систему

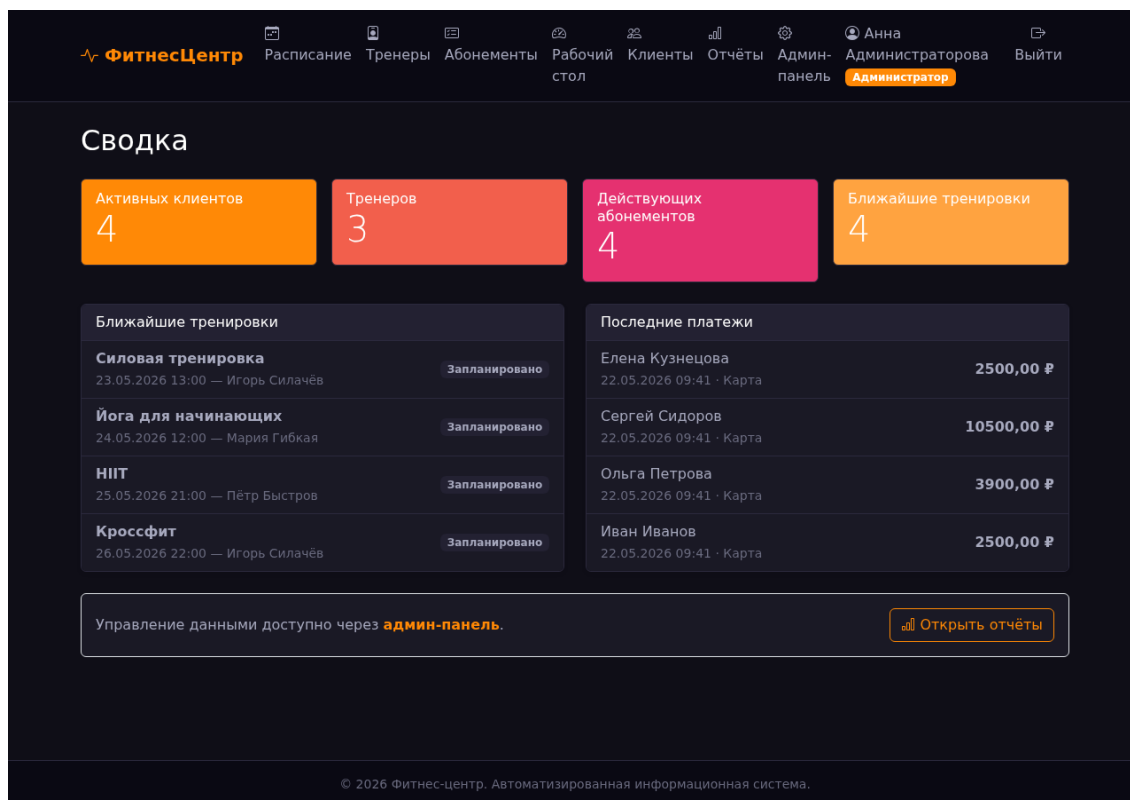


Рисунок 6 — Кабинет администратора со сводными показателями

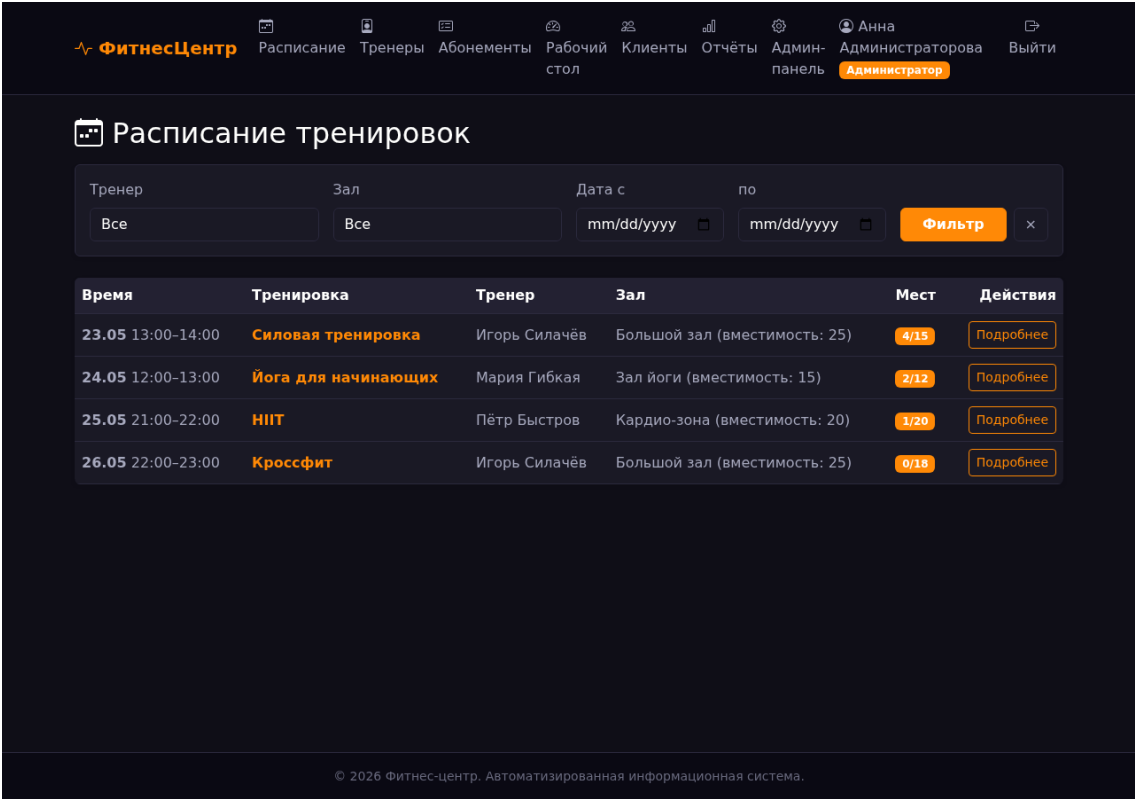


Рисунок 7 — Расписание тренировок

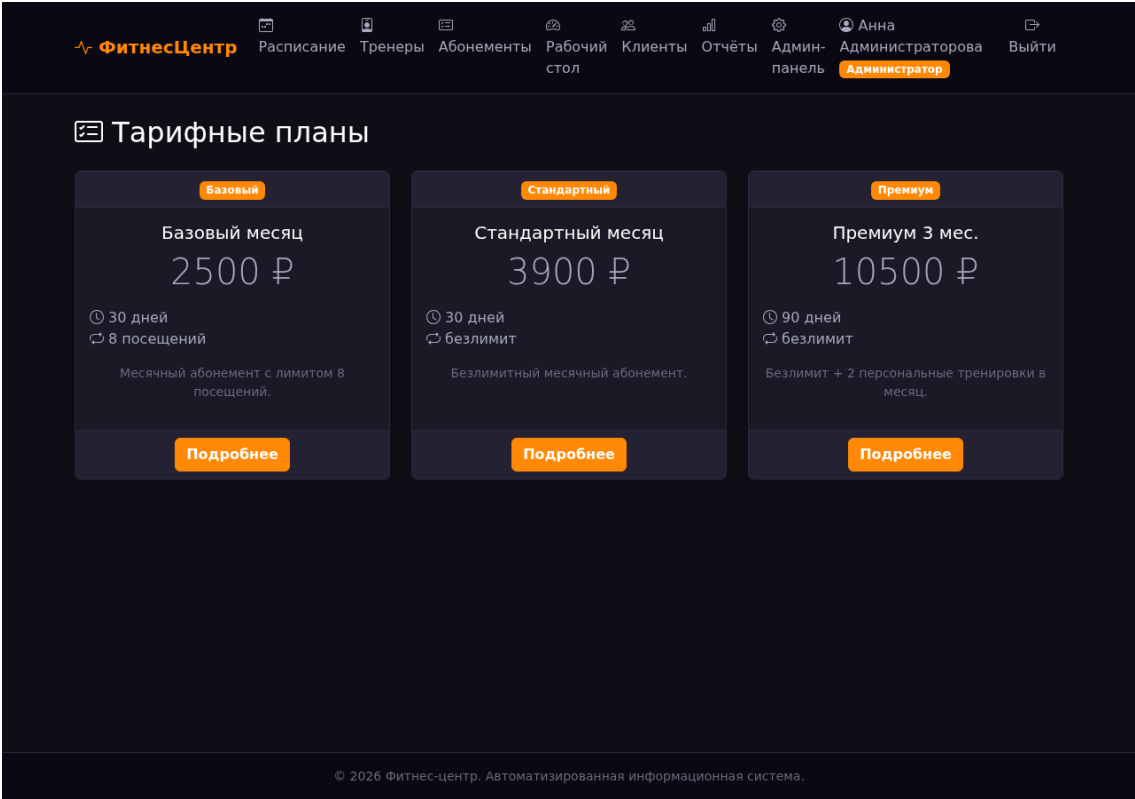


Рисунок 8 — Каталог тарифных планов абонементов

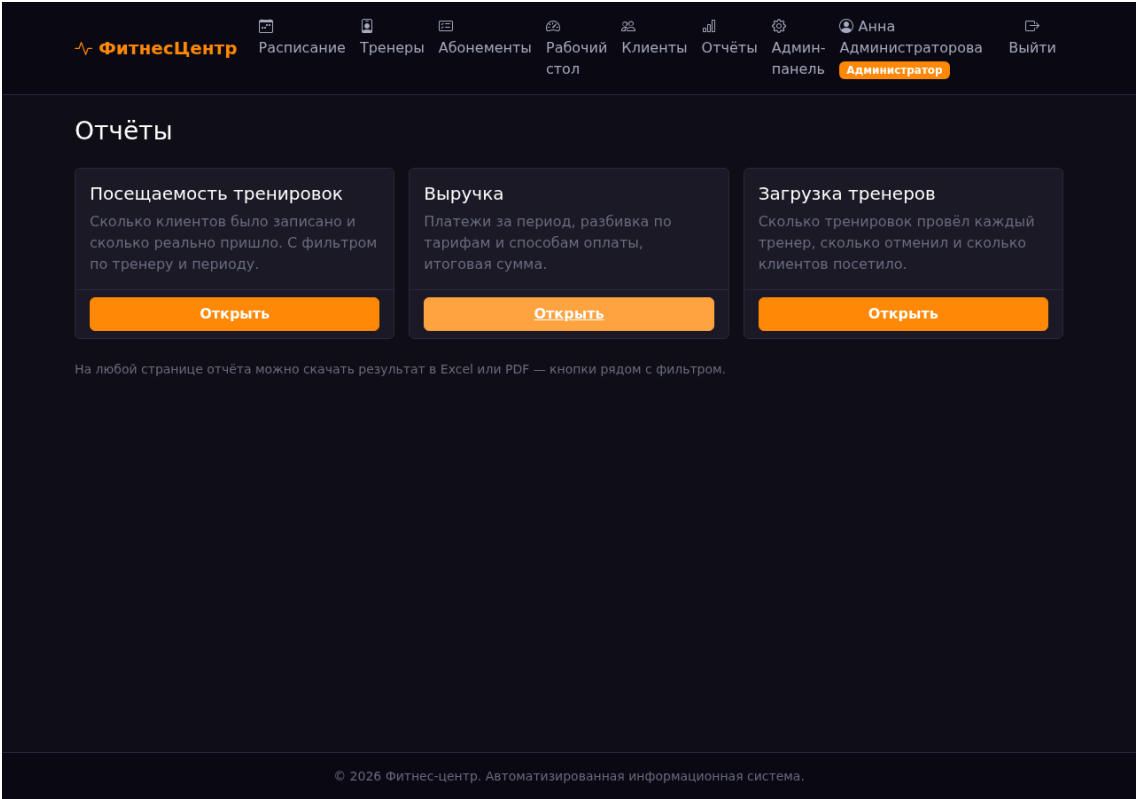


Рисунок 9 — Страница отчётов

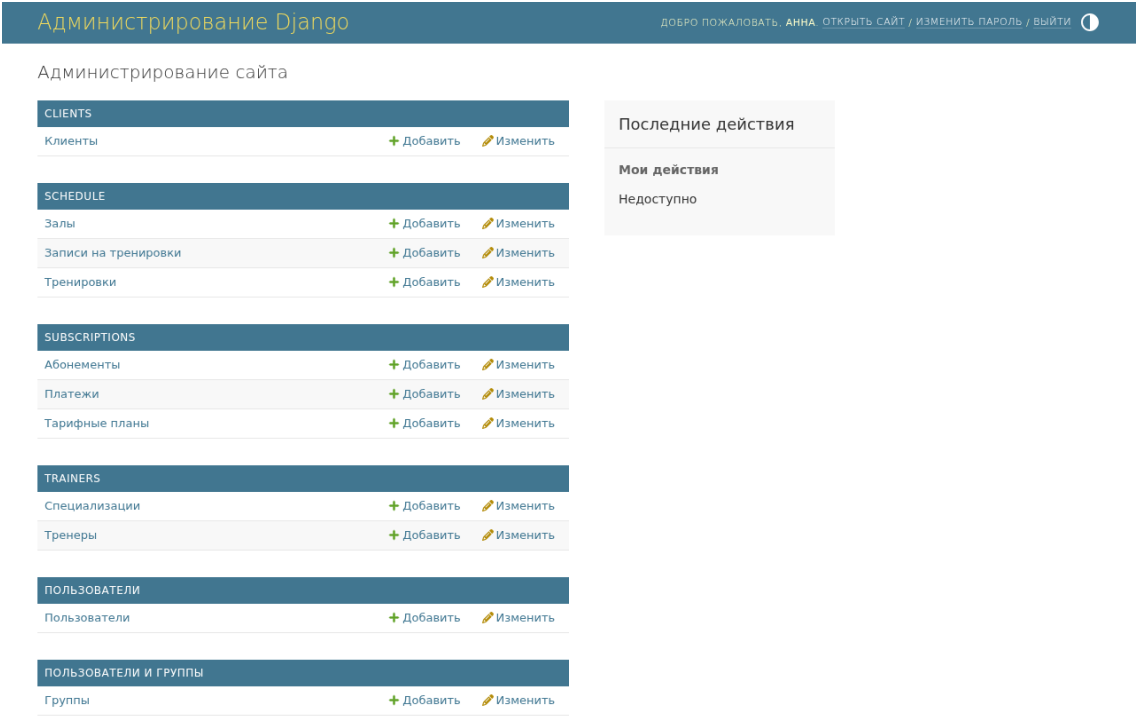


Рисунок 10 — Административная панель Django

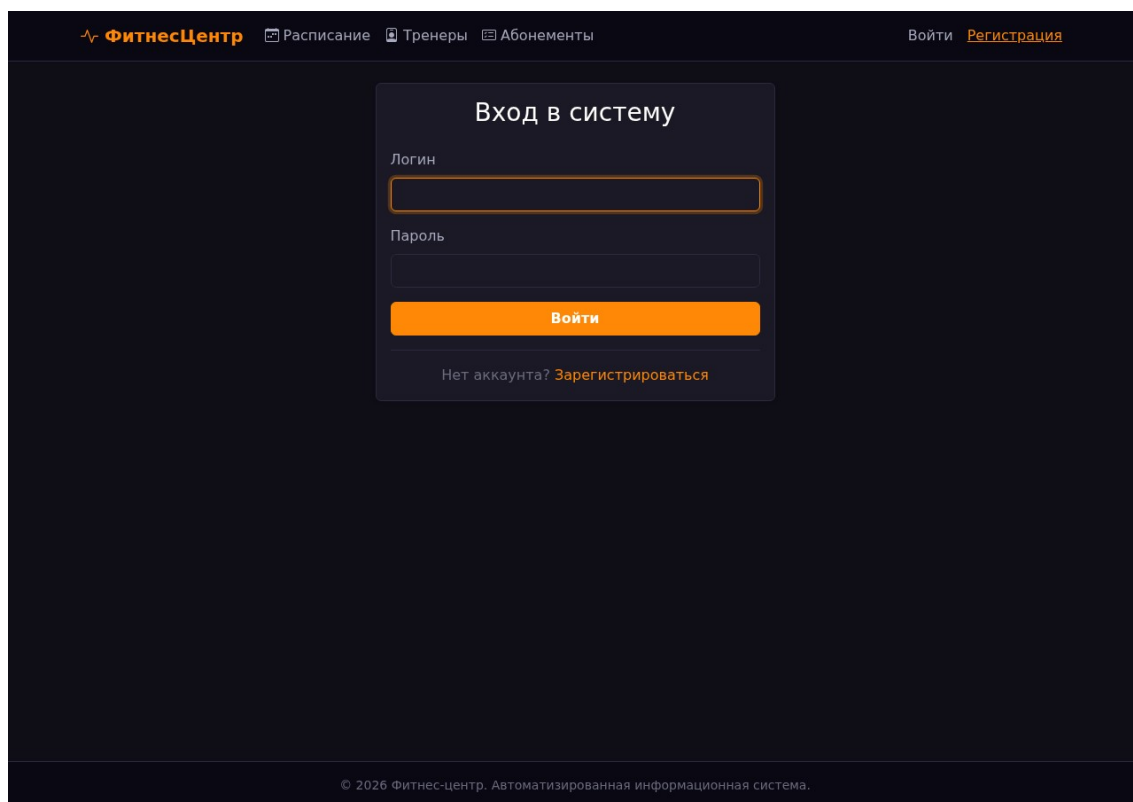


Рисунок 11 — Кабинет тренера со списком занятий

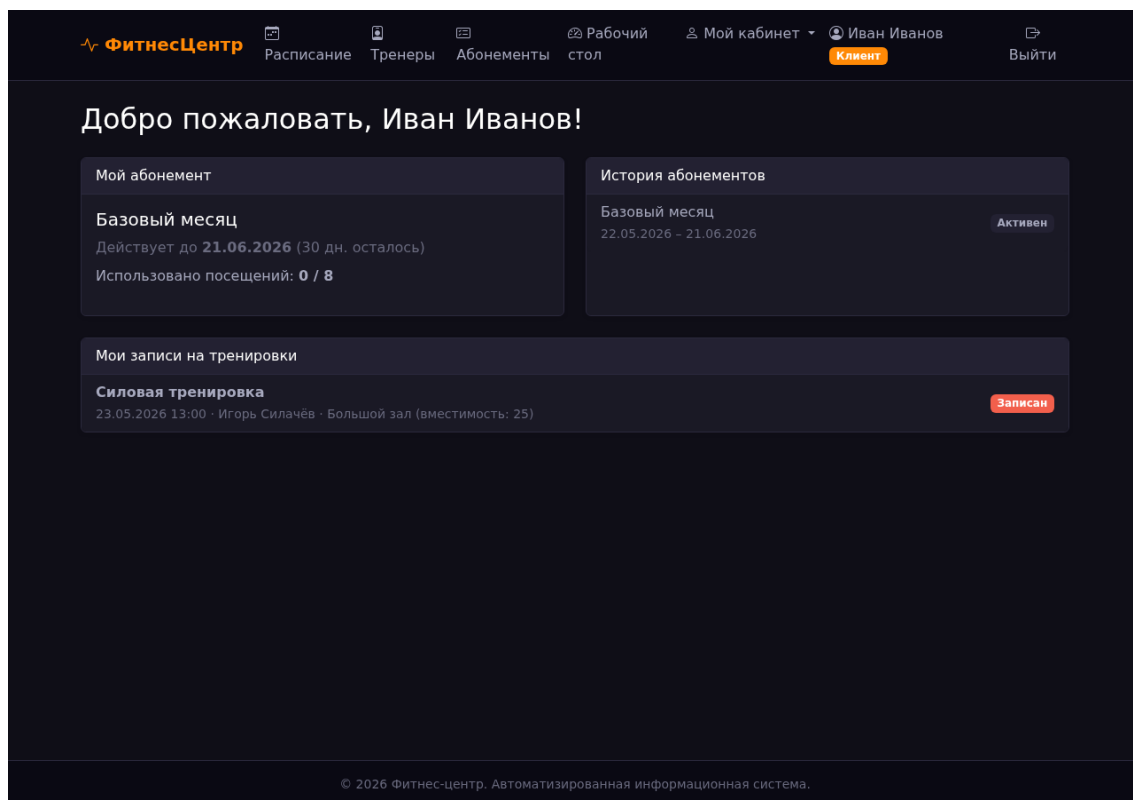


Рисунок 12 — Кабинет клиента с активным абонементом